

Практична робота №6

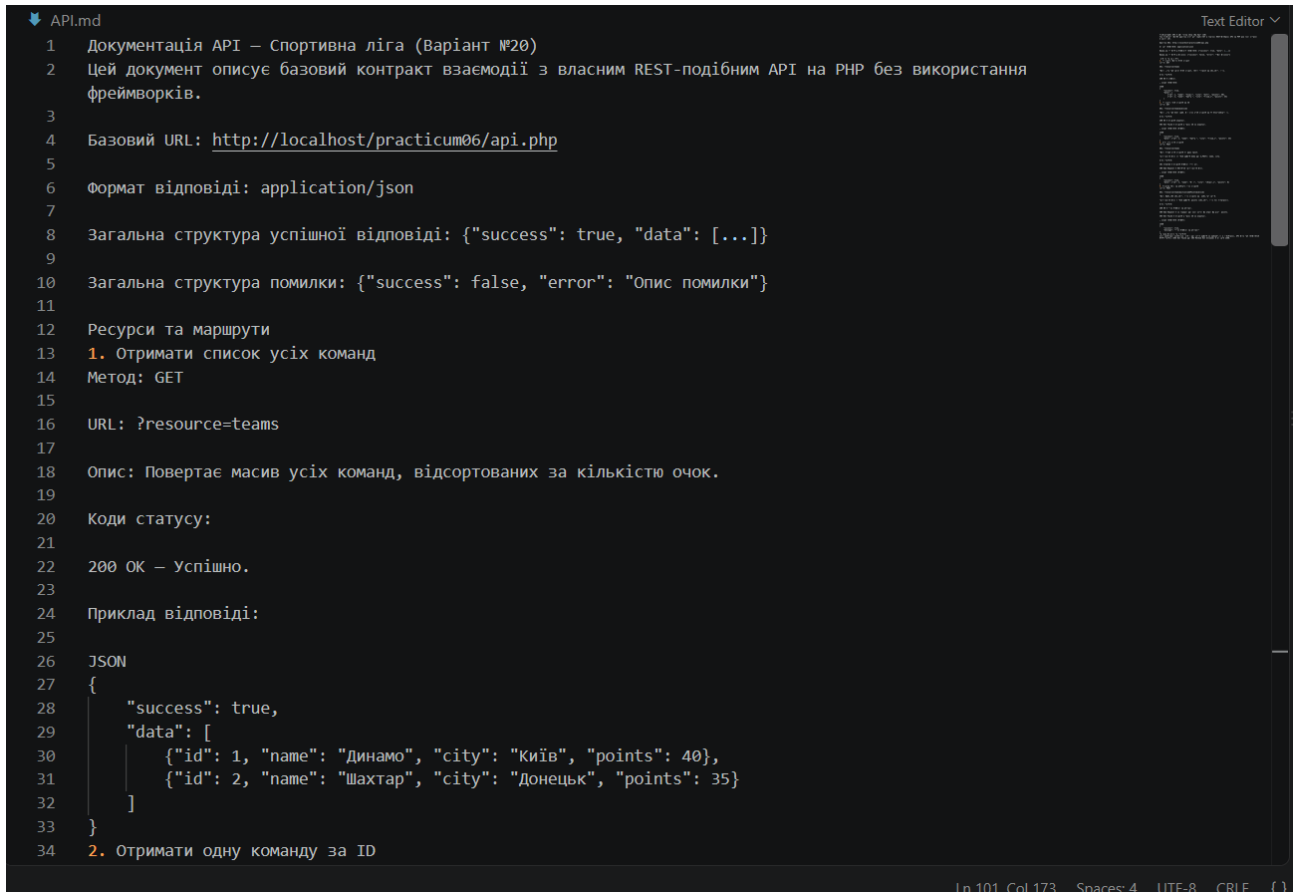
Розробка власного прикладного програмного інтерфейсу (API)

ІО-43 Череватов Артем

Варіант 20

Крок 1. Визначити контракт API свого домену

Створено файл API.md, у якому описано перелік підтримуваних запитів: HTTP-методи, ресурси, параметри та приклади JSON-відповідей для варіанта



```
API.md
1  Документація API – Спортивна ліга (Варіант №20)
2  Цей документ описує базовий контракт взаємодії з власним REST-подібним API на PHP без використання
   фреймворків.
3
4  Базовий URL: http://localhost/practicum06/api.php
5
6  Формат відповіді: application/json
7
8  Загальна структура успішної відповіді: {"success": true, "data": [...]}
9
10 Загальна структура помилки: {"success": false, "error": "Опис помилки"}
11
12 Ресурси та маршрути
13 1. Отримати список усіх команд
14 Метод: GET
15
16 URL: ?resource=teams
17
18 Опис: Повертає масив усіх команд, відсортованих за кількістю очок.
19
20 Коди статусу:
21
22 200 OK – Успішно.
23
24 Приклад відповіді:
25
26 JSON
27 {
28   "success": true,
29   "data": [
30     {"id": 1, "name": "Динамо", "city": "Київ", "points": 40},
31     {"id": 2, "name": "Шахтар", "city": "Донецьк", "points": 35}
32   ]
33 }
34 2. Отримати одну команду за ID
```

Крок 2. Створити єдину точку входу

Створено файл api.php, на початку якого одразу встановлено заголовок header('Content-Type: application/json'), а також отримано метод запиту через \$_SERVER['REQUEST_METHOD'] та параметр \$_GET['resource'].

```
header('Content-Type: application/json');

$method = $_SERVER['REQUEST_METHOD'];
$resource = $_GET['resource'] ?? null;
$id = $_GET['id'] ?? null;
$action = $_GET['action'] ?? null;
```

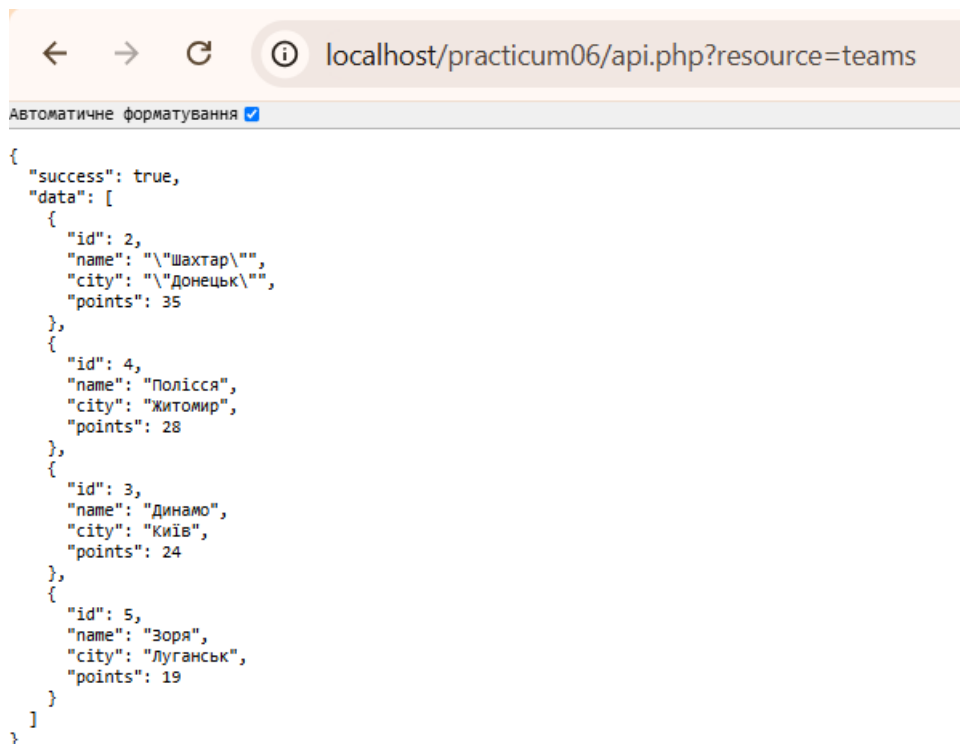
Крок 3. Реалізувати маршрутизацію

За допомогою конструкції if/elseif налаштовано перевірку: якщо прийшов запит на правильний ресурс (resource=teams), код спрямовується на відповідну обробку; для невідомого ресурсу повертається статус 404 Not Found і JSON з описом помилки.

```
11  if ($resource !== 'teams') {
12      http_response_code(404);
13      echo json_encode(['success' => false, 'error' => 'Невідомий ресурс або метод']);
14      exit;
15  }
```

Крок 4. Реалізувати GET список і одиничний запис

Налаштовано обробку GET-запитів:



Крок 5. Реалізувати POST створення запису

Реалізовано обробку POST-запиту для створення нової команди. Скрипт перевіряє наявність обов'язкових полів (name, city). Якщо чогось бракує — повертає статус 400 Bad Request; при успіху виконує INSERT через підготовлений запит і повертає статус 201 Created із даними створеного запису.

```
1  {
2      "success": true,
3      "data": {
4          "id": "6",
5          "name": "Карпати",
6          "city": "Львів",
7          "points": "15"
8      }
9  }
```

Крок 6. Реалізувати доменну дію, окрему від звичайного CRUD

Додано специфічну доменну дію для варіанта №20 через параметр action=addPoints. Метод приймає параметр points у тілі запиту і збільшує кількість очок вказаної команди в базі даних.

```
1  {
2    "success": true,
3    "message": "Очки успішно нараховано"
4  }
```

Крок 7. Обробити некоректні запити

Налаштовано коректні відповіді на помилки клієнта:

```
1  {
2    "success": false,
3    "error": "Обов'язкові поля (name, city) відсутні"
4  }
```

Крок 8. Перевірити API вручну

Проведено повне ручне тестування всіх маршрутів через браузер (для GET) та Postman / curl (для POST), під час якого перевірено відповідність HTTP-станусів і структури JSON-тіла.

